



CASE TÉCNICO — DATA & AI

Chat com Documentos via IA

Retrieval-Augmented Generation com busca semântica em Redis

Vaga	Software Engineer Pleno — Data & AI (Generalista)
Modalidade	Híbrido — Brasil
Prazo de entrega	5 dias úteis após recebimento
Entrega via	Repositório GitHub com README completo
Contato	Recrutamento Pipefy — Data & AI Team

1. Contexto do Desafio

A Pipefy é uma plataforma de automação de processos usada por mais de 4.000 empresas em 150 países. Nosso time de Data & AI está expandindo suas capacidades de inteligência artificial e busca profissionais capazes de construir produtos que conectam dados, modelos de linguagem e experiências de usuário de ponta a ponta.

Neste case, você irá construir uma solução conversacional completa que permite usuários fazerem upload de documentos e interagirem com eles via chat utilizando busca semântica e geração aumentada por recuperação (RAG — Retrieval-Augmented Generation).

Por que este desafio?

Este projeto simula situações reais do nosso dia a dia: construir pipelines de ingestão de dados, expor APIs confiáveis, integrar LLMs e entregar interfaces funcionais. Queremos avaliar sua capacidade de transitar entre todas essas camadas com autonomia e qualidade de código.

2. Objetivo

Construir uma aplicação full-stack com três componentes principais:

- 1. Upload e vetorização de documentos em uma base Redis com busca vetorial.
- 2. API conversacional (FastAPI) que responde perguntas usando RAG sobre os documentos indexados.
- 3. Interface de chat em React JS para o usuário interagir com a solução de forma intuitiva.

3. Arquitetura da Solução

A solução deve seguir a arquitetura abaixo, com cada componente containerizado via Docker:

3.1 Visão Geral dos Componentes

Camada	Tecnologias	Entregável
Frontend (React JS)	React, Vite ou CRA, Tailwind ou MUI	Chat UI com upload de arquivos e histórico de conversa
API (FastAPI)	Python 3.11+, FastAPI, LangChain	Endpoints de ingestão e chat com RAG
Vector Store	Redis Stack (RedisSearch)	Armazenamento e busca de embeddings

Camada	Tecnologias	Entregável
LLM	OpenAI GPT-4o ou Claude Sonnet (Anthropic)	Geração de respostas contextualizadas
Orquestração	LangChain / LangGraph	Pipeline de ingestão + chain de RAG
Infraestrutura	Docker Compose	Todos os serviços containerizados

4. Requisitos Técnicos

4.1 Camada de Dados — Ingestão e Vetorização

O serviço de ingestão deve processar documentos e armazená-los como vetores no Redis:

- Aceitar arquivos nos formatos **PDF e TXT** (mínimo obrigatório).
- Fazer chunking do texto com overlap configurável (ex: chunks de 500 tokens, overlap de 50).
- Gerar embeddings utilizando a API OpenAI (**text-embedding-ada-002** ou **text-embedding-3-small**) ou via Sentence Transformers (open-source).
- Armazenar os vetores no Redis Stack utilizando o módulo RedisSearch com índice vetorial (HNSW ou FLAT).
- Persistir metadados relevantes: nome do arquivo, data de upload, número do chunk, texto original do chunk.

```
# Exemplo de estrutura esperada no Redis
doc:{file_id}:chunk:{n} = {
  'content':      '...texto do chunk...',
  'embedding':    [...vetor float32...],
  'source':       'relatorio_q3.pdf',
  'chunk_index':  3,
  'uploaded_at':  '2025-05-08T10:00:00Z'
}
```

4.2 API — FastAPI com RAG

A API deve expor pelo menos os seguintes endpoints:

Método	Endpoint	Descrição	Resposta
POST	/upload	Recebe arquivo(s) e dispara pipeline de ingestão	{ file_id, chunks_indexed, status }
GET	/documents	Lista documentos indexados na base	[[{ file_id, name, uploaded_at, chunks }]]
DELETE	/documents/{id}	Remove documento e seus vetores do Redis	{ deleted: true }

Método	Endpoint	Descrição	Resposta
POST	/chat	Recebe pergunta e retorna resposta com RAG	{ answer, sources: [...] }
GET	/health	Health check da API e conectividade Redis	{ status: 'ok', redis: 'connected' }

Requisitos adicionais da API:

- O endpoint /chat deve implementar o fluxo RAG completo: embedding da query → busca vetorial no Redis → construção do prompt com contexto → chamada ao LLM → retorno da resposta com as fontes utilizadas.
- Utilizar **LangChain** para orquestrar o pipeline de RAG (RetrievalQA ou LCEL — LangChain Expression Language).
- Suporte a histórico de conversa por sessão (armazenar últimas N mensagens para contexto).
- Respostas do chat devem incluir o campo sources com os trechos e documentos utilizados.
- Variáveis de ambiente para configuração (chaves de API, URL do Redis, modelo LLM).

```
# Exemplo de payload — POST /chat
{
  'question': 'Quais foram os principais resultados do Q3?',
  'session_id': 'abc-123',
  'top_k': 5
}

# Resposta esperada
{
  'answer': 'Os principais resultados do Q3 foram...',
  'sources': [
    { 'chunk': '...trecho do doc...', 'source': 'relatorio_q3.pdf', 'score': 0.91 }
  ],
  'session_id': 'abc-123'
}
```

4.3 Frontend — Chat UI em React JS

A interface deve ser funcional e clara. Não é necessário design sofisticado, mas espera-se usabilidade e organização do código:

- Tela de upload de arquivos com drag-and-drop ou seletor de arquivo.
- Indicador de progresso durante o processo de ingestão.
- Lista de documentos indexados com opção de remoção.
- Interface de chat com histórico de mensagens (usuário e assistente).
- Exibição das fontes utilizadas em cada resposta (collapse ou tooltip).

- Campo de input com suporte a envio por Enter.
- Indicador de carregamento enquanto a API processa a resposta.

Tecnologias sugeridas para o Frontend

React JS com Vite ou Create React App.

Gerenciamento de estado: Context API, Zustand ou Redux Toolkit (à sua escolha).

Estilização: Tailwind CSS, Material UI, Chakra UI ou CSS Modules — livre escolha.

Chamadas HTTP: Axios ou Fetch API nativo.

Não é obrigatório SSR (Next.js) — uma SPA simples é suficiente.

4.4 Infraestrutura — Docker e Docker Compose

Todos os serviços devem ser containerizados e orquestrados via Docker Compose:

- Dockerfile para o serviço de API (FastAPI).
- Dockerfile para o Frontend (build estático servido via Nginx ou node).
- **docker-compose.yml** com os serviços: api, frontend, redis (redis/redis-stack:latest).
- Variáveis de ambiente via arquivo **.env** (não commitar chaves de API).
- Volumes para persistência dos dados do Redis.
- Health checks e **depends_on** configurados corretamente.

```
# Estrutura mínima esperada do docker-compose.yml
services:
  redis:
    image: redis/redis-stack:latest
    ports: ['6379:6379', '8001:8001']
    volumes: [redis_data:/data]

  api:
    build: ./backend
    ports: ['8000:8000']
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - REDIS_URL=redis://redis:6379
    depends_on: [redis]

  frontend:
    build: ./frontend
    ports: ['3000:3000']
    depends_on: [api]

volumes:
  redis_data:
```

4.5 LangGraph — Orquestração de Agente (Diferencial)

Como diferencial, implemente o pipeline de RAG utilizando LangGraph ao invés de chains simples do LangChain. O grafo deve conter pelo menos os seguintes nós:

- **retriever_node**: busca semântica no Redis e recuperação dos chunks relevantes.
- **context_builder_node**: monta o prompt com o contexto recuperado e o histórico da conversa.
- **llm_node**: chamada ao modelo de linguagem com o prompt montado.
- **response_formatter_node**: formata a resposta final incluindo as fontes.

O uso de LangGraph permite maior controle sobre o fluxo, facilita a adição de nós de validação, retry e branching condicional — padrão que utilizamos internamente na Pipefy.

5. Testes Unitários

A cobertura de testes é um critério de avaliação importante. Utilize pytest como framework principal:

- Testes dos serviços de processamento: chunking, geração de embeddings (mock da API), indexação no Redis.
- Testes dos endpoints FastAPI utilizando **httpx** e **TestClient**.
- Testes do pipeline RAG com mocks para o Redis e para o LLM (evitar chamadas reais de API nos testes).
- Cobertura mínima esperada: 60% do código de backend.
- Inclua um **Makefile** ou script de conveniência: **make test** ou **./run_tests.sh**.

```
# Estrutura de testes esperada
tests/
├── test_ingestion.py      # chunking, embeddings, redis indexing
├── test_api_upload.py     # endpoint POST /upload
├── test_api_chat.py       # endpoint POST /chat (RAG pipeline)
├── test_api_documents.py  # endpoints GET e DELETE /documents
└── conftest.py           # fixtures: redis mock, llm mock, test client
```

6. Critérios de Avaliação

O case será avaliado pelos seguintes critérios, em ordem de peso:

Critério	Descrição	Peso
Funcionalidade	A solução funciona end-to-end: upload → indexação → chat com resposta contextualizada.	Alta — obrigatório
Qualidade de Código	Organização, legibilidade, tipagem Python (type hints), separação de responsabilidades.	Alta

Critério	Descrição	Peso
RAG Pipeline	Qualidade da implementação do pipeline de busca semântica e geração de resposta.	Alta
Testes	Cobertura e qualidade dos testes unitários com mocks adequados.	Média-Alta
Docker & Infra	Configuração correta do Docker Compose, variáveis de ambiente, persistência.	Média
LangGraph	Uso correto do LangGraph para orquestrar o pipeline de RAG.	Diferencial
Frontend UX	Clareza e usabilidade da interface — não é esperado design profissional.	Média
README	Documentação clara: como rodar, variáveis necessárias, decisões de design.	Média
Diferenciais	Streaming, multi-documento, deploy em nuvem, CI com GitHub Actions.	Bônus

7. Estrutura Esperada do Repositório

my-rag-app/	
backend/	
app/	
main.py	# FastAPI app e rotas
routers/	
upload.py	# POST /upload, DELETE /documents/{id}
chat.py	# POST /chat
documents.py	# GET /documents
services/	
ingestion.py	# chunking + embedding + redis indexing
retriever.py	# busca semântica no Redis
rag_chain.py	# LangChain/LangGraph RAG pipeline
models/	
schemas.py	# Pydantic models
config.py	# configurações via env vars
tests/	
...	# pytest tests
Dockerfile	
requirements.txt	
frontend/	
src/	
components/	
ChatWindow.jsx	
FileUpload.jsx	
DocumentList.jsx	
App.jsx	
Dockerfile	
package.json	
docker-compose.yml	
.env.example	
Makefile	
README.md	

8. Instruções de Entrega

8.1 O que entregar

- Repositório público no GitHub com todo o código-fonte.
- **README.md** completo com: descrição do projeto, como rodar localmente (passo a passo), variáveis de ambiente necessárias (.env.example), decisões de arquitetura e trade-offs, e instruções para rodar os testes.
- O projeto deve rodar com um único comando: **docker-compose up --build**.

8.2 O que NÃO entregar

- Não commite chaves de API reais no repositório.
- Não é necessário deploy em produção, mas é um diferencial.

8.3 Prazo

Prazo de entrega

5 dias úteis a partir do recebimento deste documento.

Envie o link do repositório para o e-mail do recrutador com o assunto:

[Case] Software Engineer Data & AI — [Seu Nome]

9. Dicas e Orientações

9.1 Redis Vector Search

Utilize o redis-py com o módulo redis-stack. A imagem Docker redis/redis-stack:latest já inclui o RedisSearch. Defina o índice com o tipo de métrica COSINE e dimensão compatível com o modelo de embedding escolhido (ex: 1536 para text-embedding-ada-002 da OpenAI).

```
from redis import Redis
from redis.commands.search.field import VectorField, TextField
from redis.commands.search.indexDefinition import IndexDefinition, IndexType

r = Redis(host='redis', port=6379)
schema = (
    TextField('content'),
    TextField('source'),
    VectorField('embedding', 'HNSW', {
        'TYPE': 'FLOAT32',
        'DIM': 1536,
        'DISTANCE_METRIC': 'COSINE'
    })
)
r.ft('docs').create_index(schema, definition=IndexDefinition(
```



```
prefix=['doc:'], index_type=IndexType.HASH
))
```

9.2 Pipeline RAG — Fluxo básico

4. Receber a pergunta do usuário.
5. Gerar o embedding da pergunta usando o mesmo modelo utilizado na ingestão.
6. Executar busca KNN no Redis para recuperar os top-K chunks mais similares.
7. Construir o prompt incluindo os chunks como contexto e o histórico da conversa.
8. Chamar o LLM (OpenAI ou Anthropic) com o prompt montado.
9. Retornar a resposta com as fontes utilizadas.

9.3 Modelos de LLM suportados

- **OpenAI:** gpt-4o, gpt-4o-mini, gpt-3.5-turbo.
- **Anthropic:** claude-3-5-sonnet, claude-3-haiku.
- **Open-source (diferencial):** Ollama local com llama3, mistral ou gemma.

9.4 Soluções Open-Source aceitas

Caso não tenha créditos em APIs pagas, a solução pode ser 100% open-source:

- **Embeddings:** sentence-transformers (all-MiniLM-L6-v2) — roda localmente, sem custo.
- **LLM:** Ollama com modelos locais (llama3:8b, mistral:7b).
- O importante é que a arquitetura RAG funcione corretamente.

10. Implementações de Diferencial

Não obrigatórias, mas valorizadas na avaliação:

- Streaming de respostas (Server-Sent Events ou WebSocket) para o frontend exibir a resposta em tempo real.
- Suporte a múltiplos arquivos simultâneos no mesmo contexto de chat.
- Remoção de documentos com limpeza dos vetores correspondentes no Redis.
- Deploy em nuvem (GCP Cloud Run, Railway, Render ou similar) com link funcional.
- Pipeline de CI com GitHub Actions executando os testes a cada push.
- Uso de LangGraph para o pipeline de RAG ao invés de chains simples.
- Suporte a arquivos DOCX além de PDF e TXT.
- Interface com suporte a múltiplas sessões de chat com nomes customizáveis.

11. Dúvidas

Caso tenha dúvidas sobre o escopo ou requisitos do case, entre em contato com o time de recrutamento antes de iniciar a implementação. Perguntas técnicas sobre decisões de arquitetura são bem-vindas e também fazem parte da avaliação — demonstram raciocínio e capacidade de comunicação.



Boa sorte — estamos ansiosos para ver o que você vai construir.